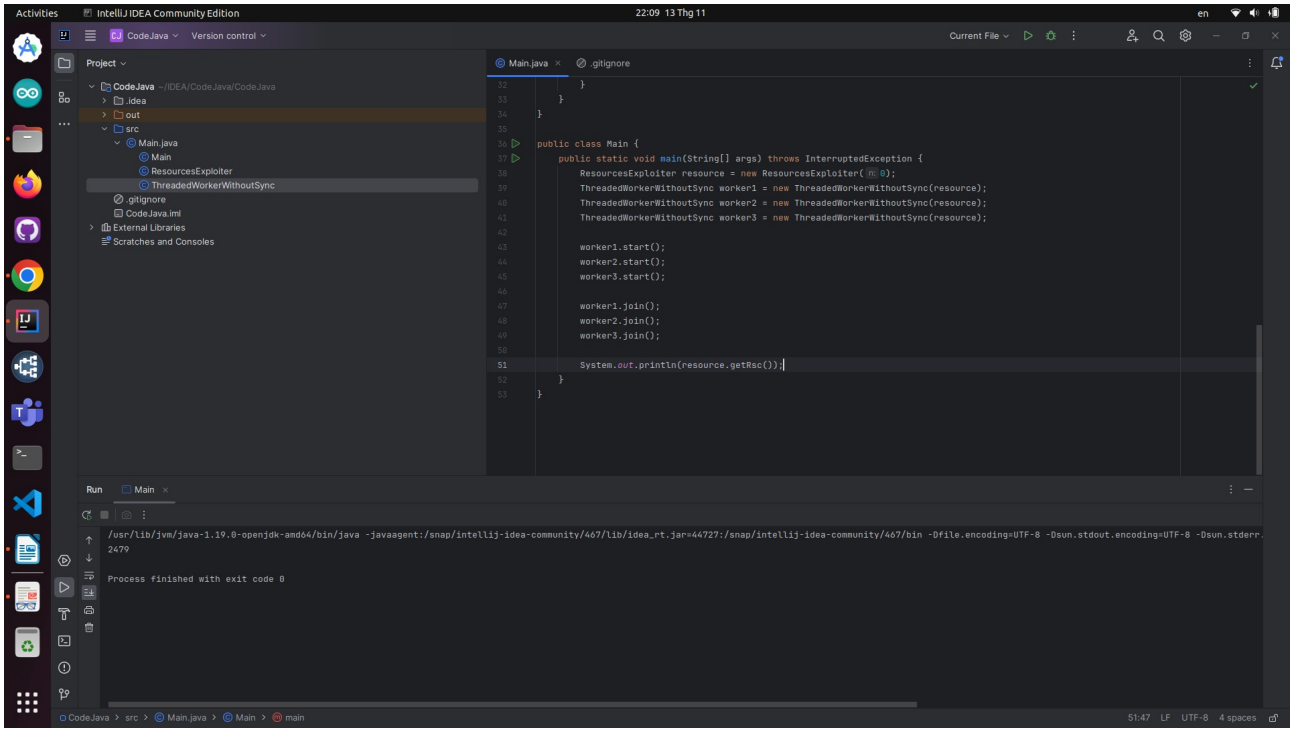
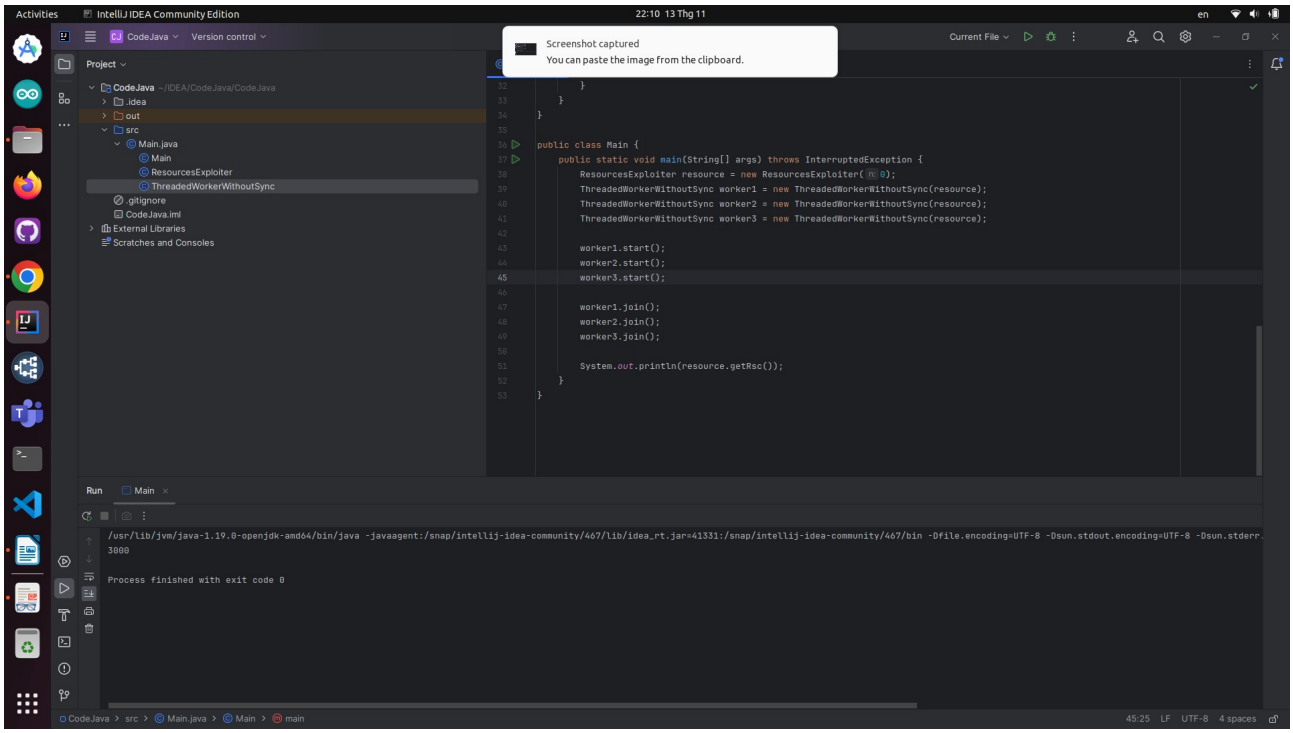
BÀI THỰC HÀNH
MÔN HỌC: HỆ PHÂN TÁN
CHƯƠNG 4: ĐỒNG BỘ HÓA

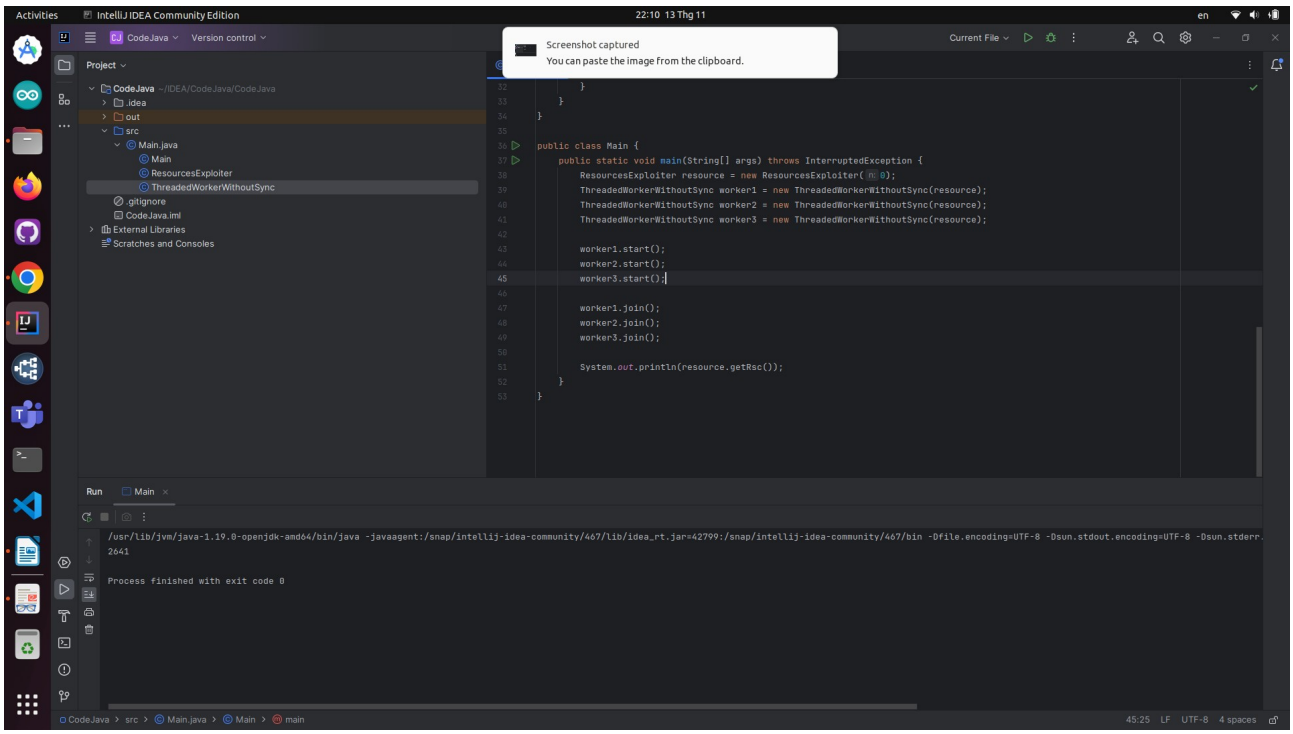
HỌ TÊN SV: Trần Ngọc Bảo
MSSV: 20215529

MÃ LỚP: 144944
MÃ HỌC PHẦN: IT4611

Câu hỏi 1: Chạy chương trình trên vài lần. Bạn nhận thấy điều gì Giải thích!

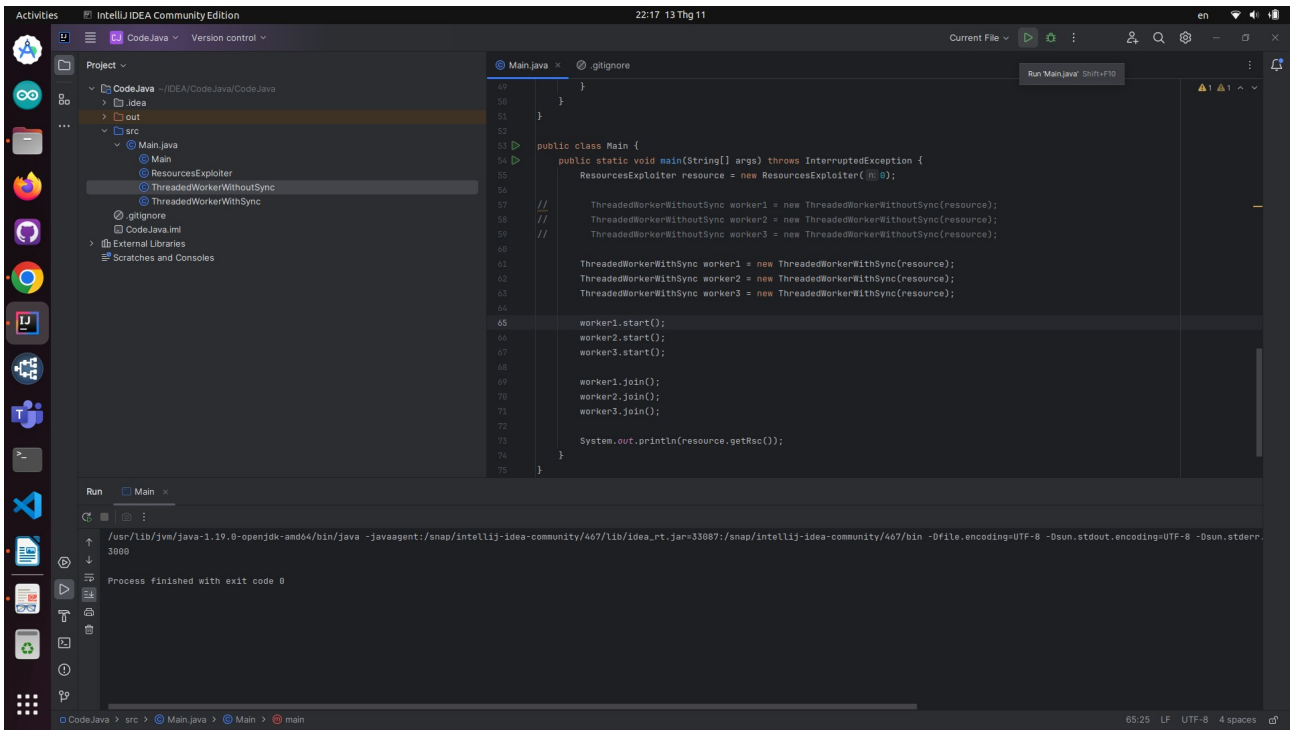


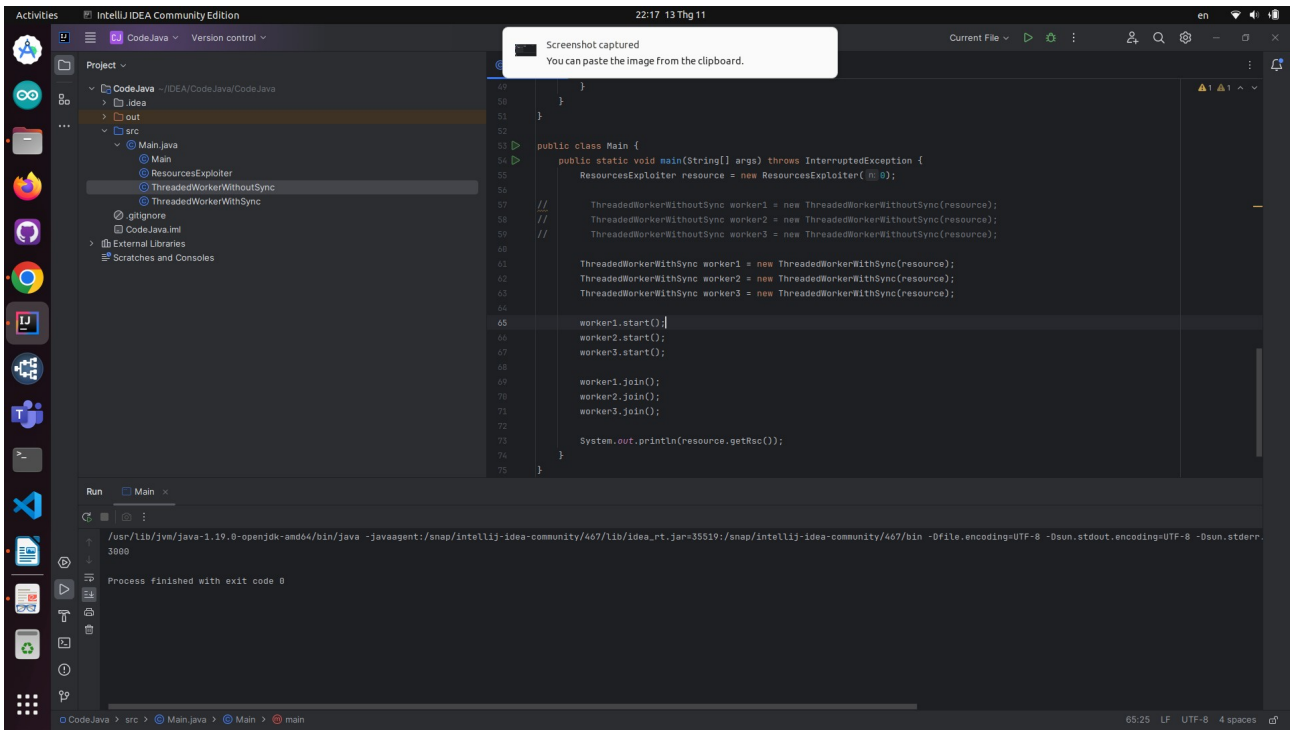


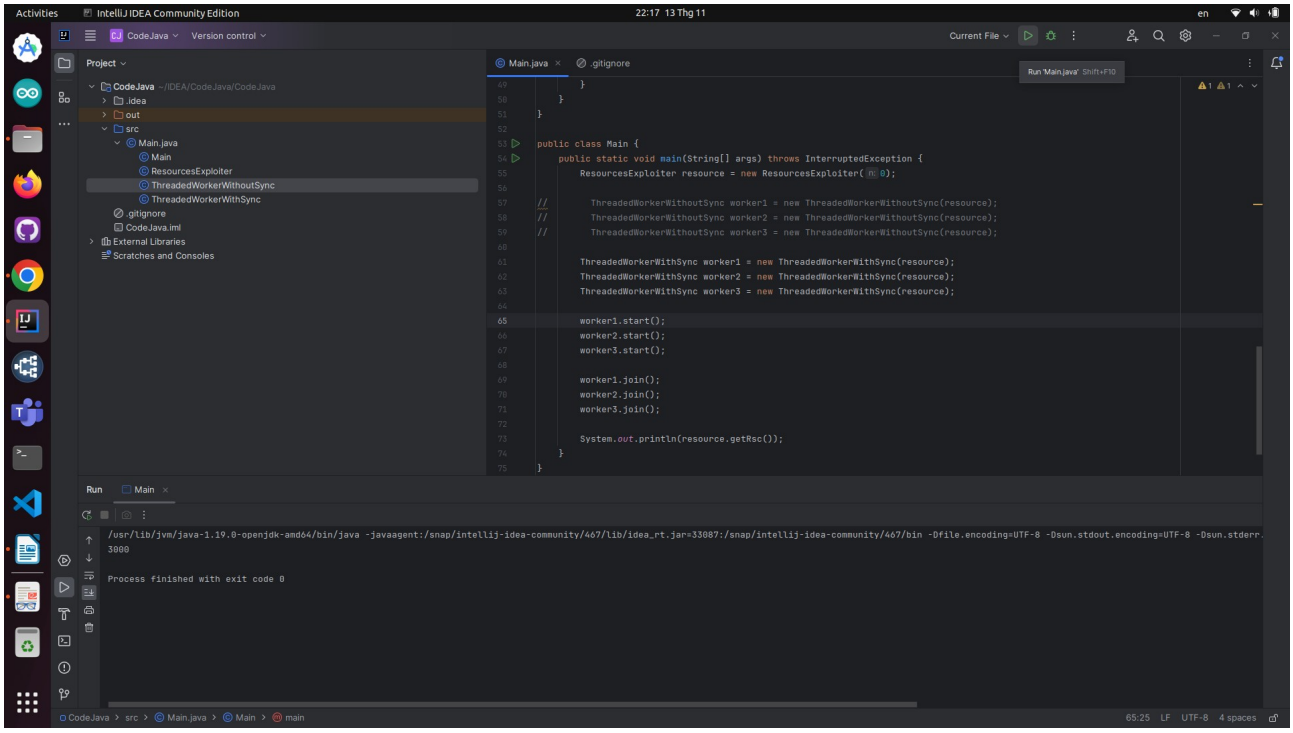


Nhận thấy mỗi lần chạy đều ra kết quả khác nhau. Vì: 3 worker đều sử dụng chung tài nguyên găng là resource nên xảy ra xung đột khi gọi hàm exploit().

Câu hỏi 2: Thay đổi đoạn mã trong chương trình chạy (phương thức main), thay đổi kiểu của 3 thực thể worker1-3 thành ThreadedWorkerWithSync. Bạn nhận thấy sự thay đổi gì ở đầu ra khi chạy chương trình đó khi so sánh với câu hỏi 1? Giải thích!

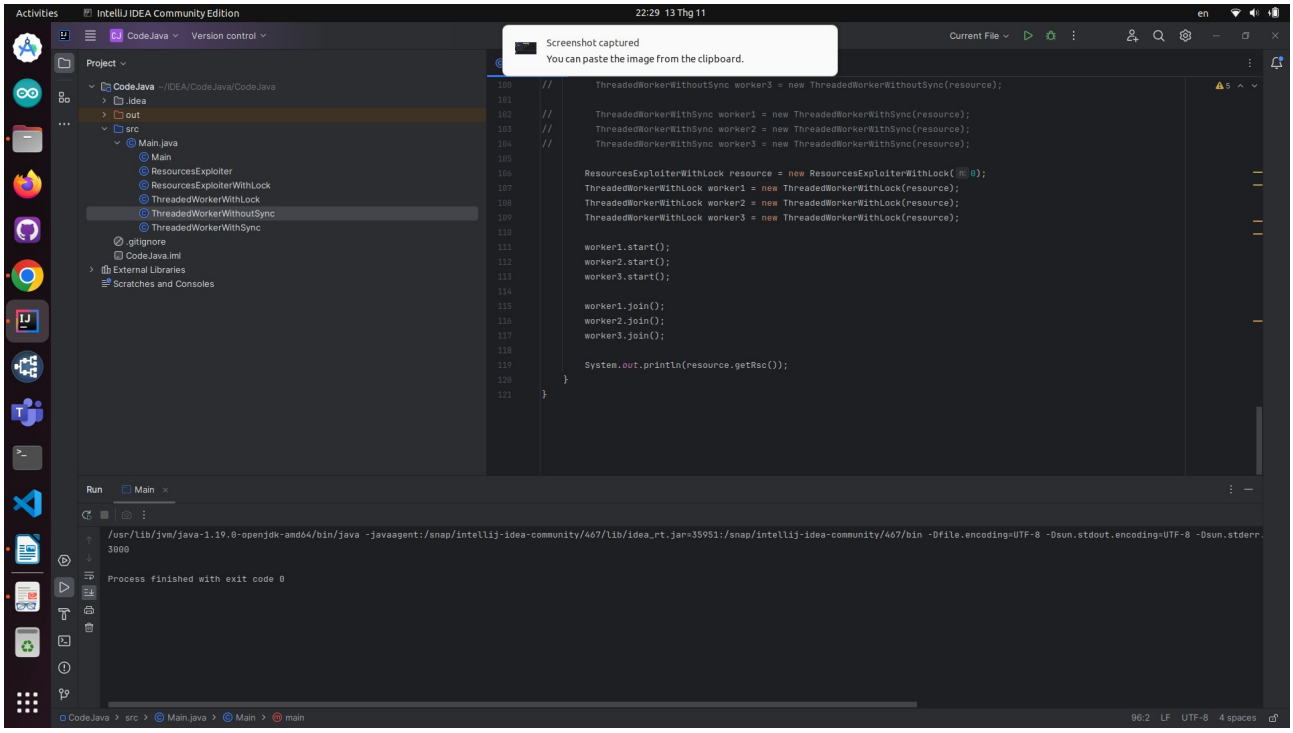


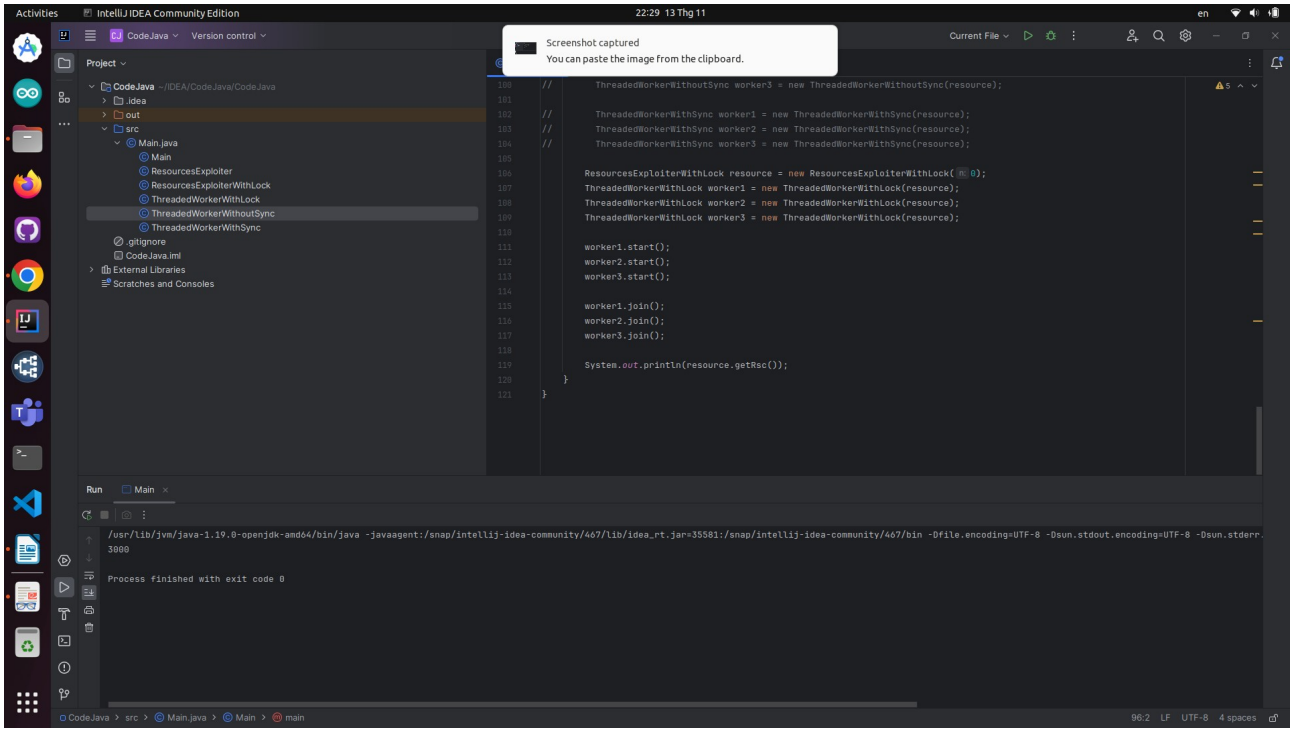


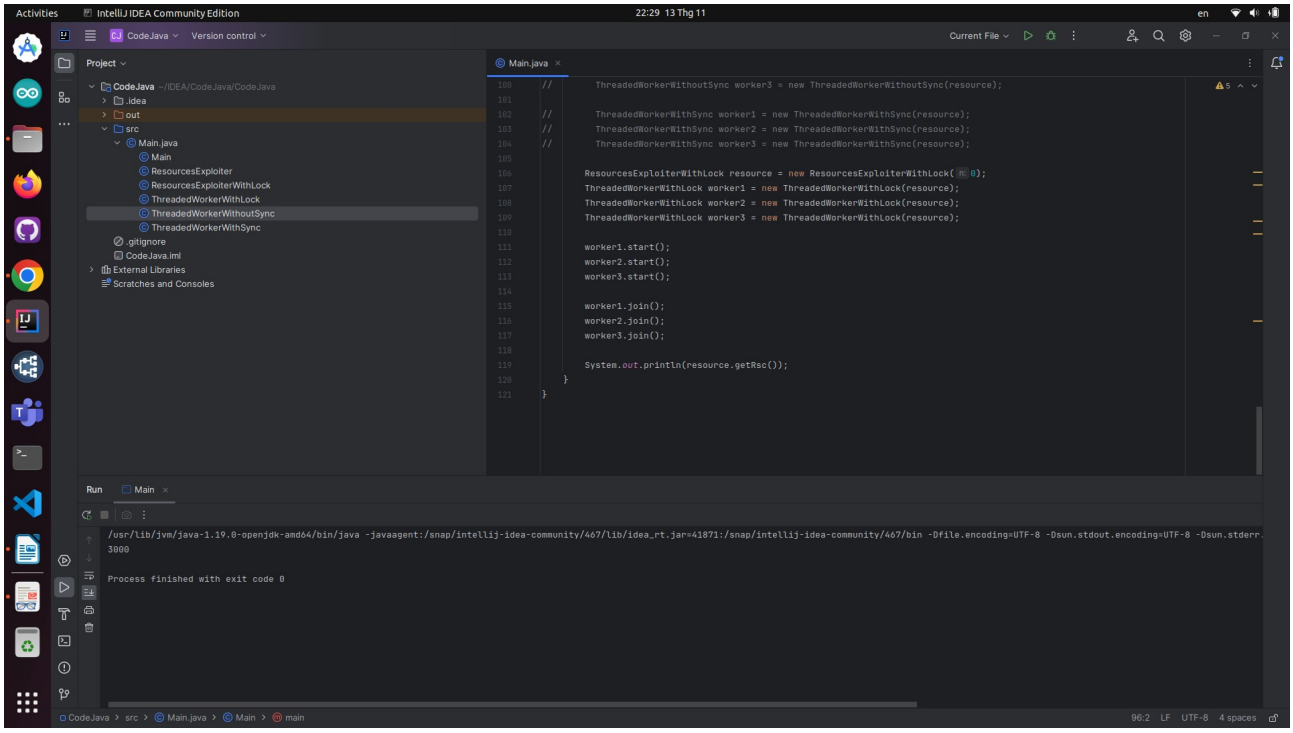


Nhận thấy cả 3 lần chạy đều cho ra chung 1 kết quả là 3000. Vì: đã áp dụng synchronized ở biến rExp cho nên 3 worker đã được đồng bộ.

Câu hỏi 3: Thay đổi đoạn code của chương trình chạy chính bằng cách thay thế kiểu của 3 thực thể worker1-3 thành ThreadedWorkerWithLock. Có khác nhau gì so với đầu ra của câu hỏi 1. Giải thích!





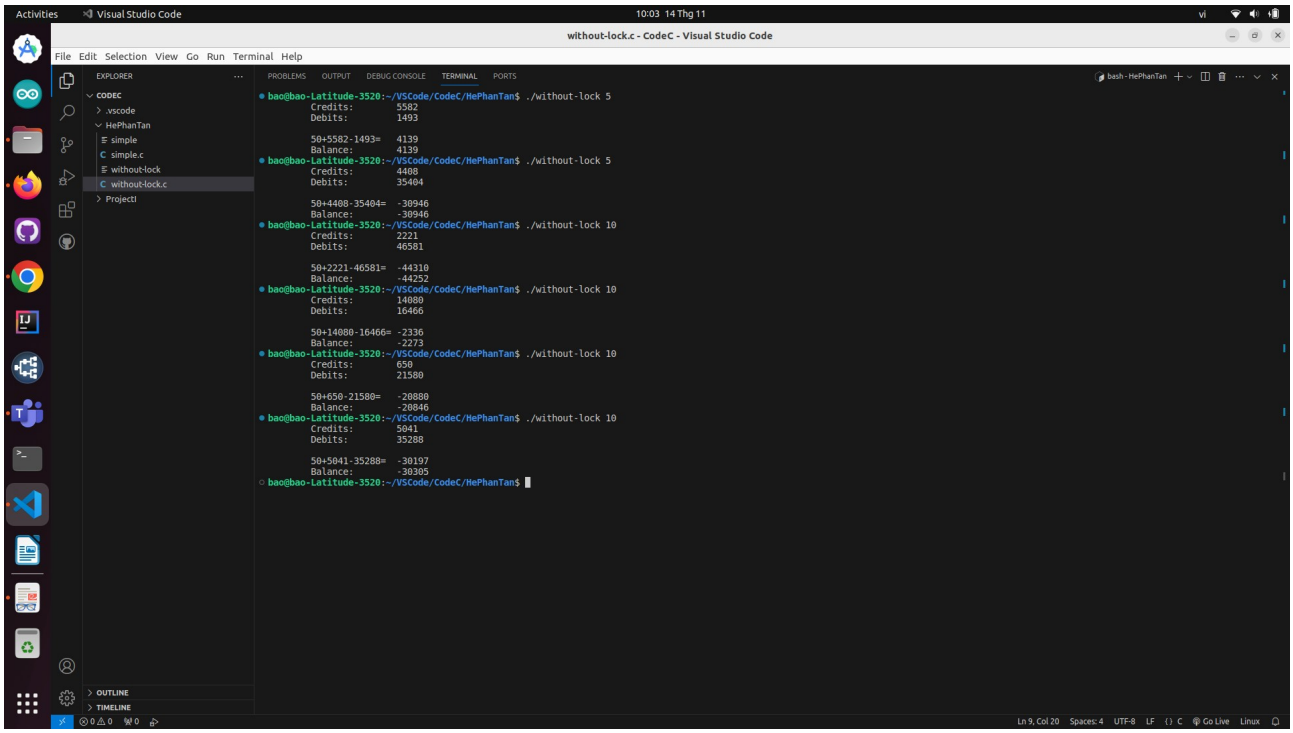


Nhận thấy kết quả giống câu 2. Vì: 3 worker cũng đã được đồng bộ bằng cơ chế Lock: khi 1 worker sử dụng tài nguyên nó sẽ lock lại không cho các worker còn lại sử dụng.

Câu hỏi 4: Hoàn thiện file trên (điền vào phần YOUR-CODE-HERE) với một vòng lặp để tăng biến shared lên một đơn vị trong vòng 5 giây. (gợi ý: hàm time(NULL) sẽ trả về giá trị thời gian của hệ thống với đơn vị là giây).

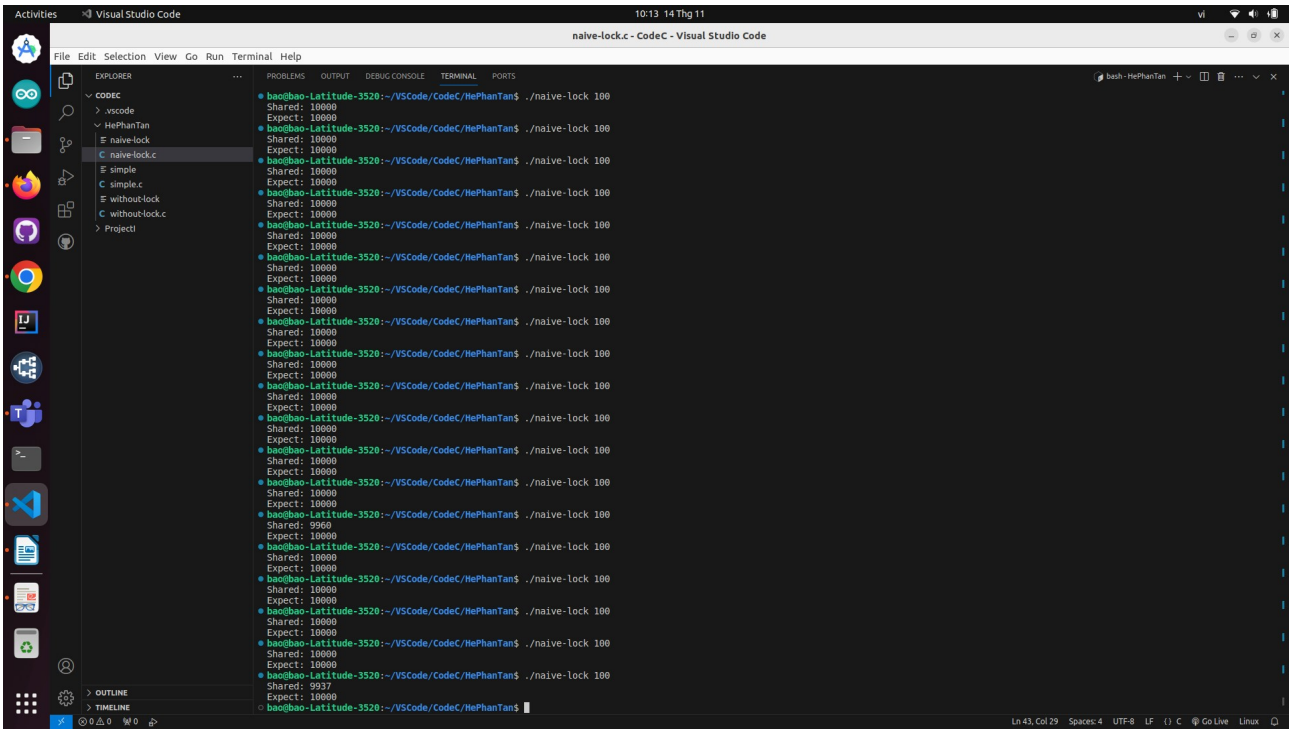
```
while (time(NULL) != end);
++shared;
```

Câu hỏi 5: Bây giờ hãy tăng giá trị số luồng và giá trị của số lần giao dịch NUM_TRANS sau mỗi lần chạy chương trình cho đến khi nào bạn thấy sự khác nhau giữa giá trị Balance (giá trị còn lại trong tài khoản) và INIT_BALANCE+credits-debits. Giải thích tại sao lại có sự khác nhau đó.



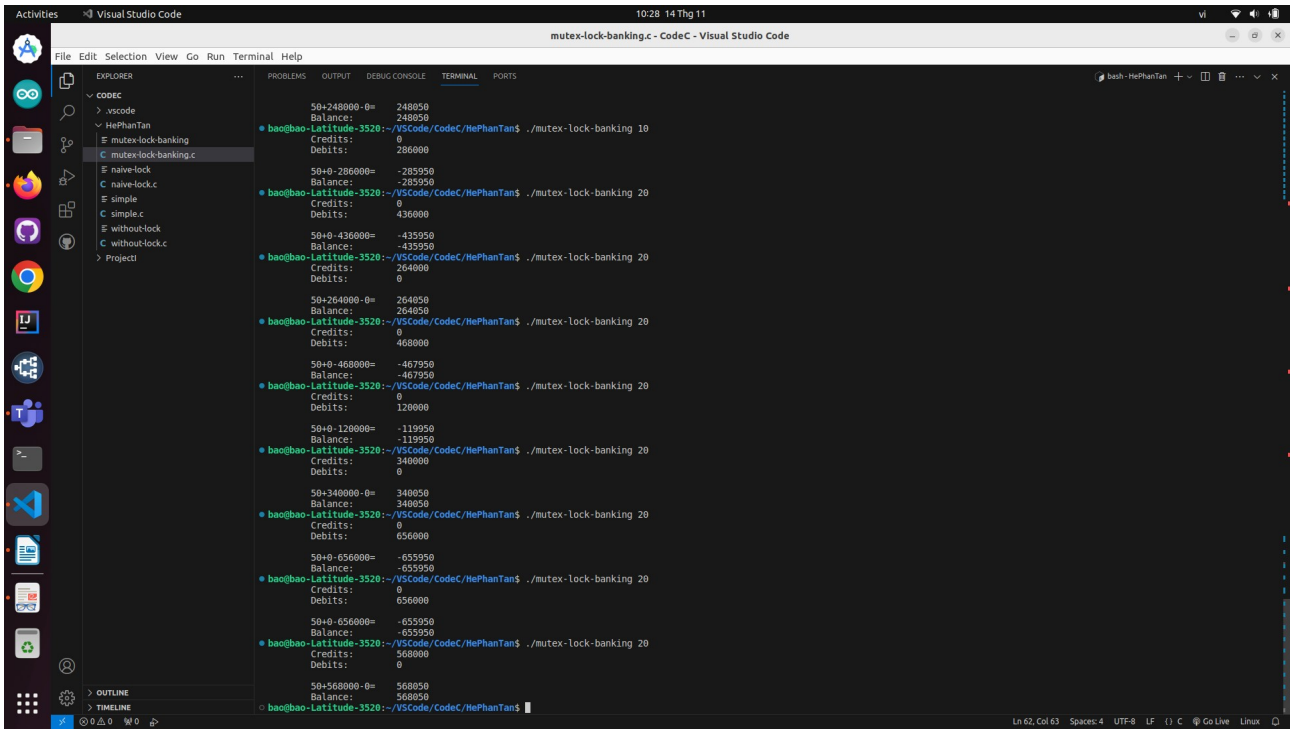
Vì: khi tăng giá trị luồng và giá trị của số lần giao dịch thì xác suất xảy ra tương tranh cao hơn.

Câu hỏi 6: Hãy build và chạy chương trình này. Chạy lặp đi lặp lại đến bao giờ bạn thấy sự khác nhau giữa 2 giá trị Shared và Expect. Phân tích mã nguồn để hiểu vấn đề.



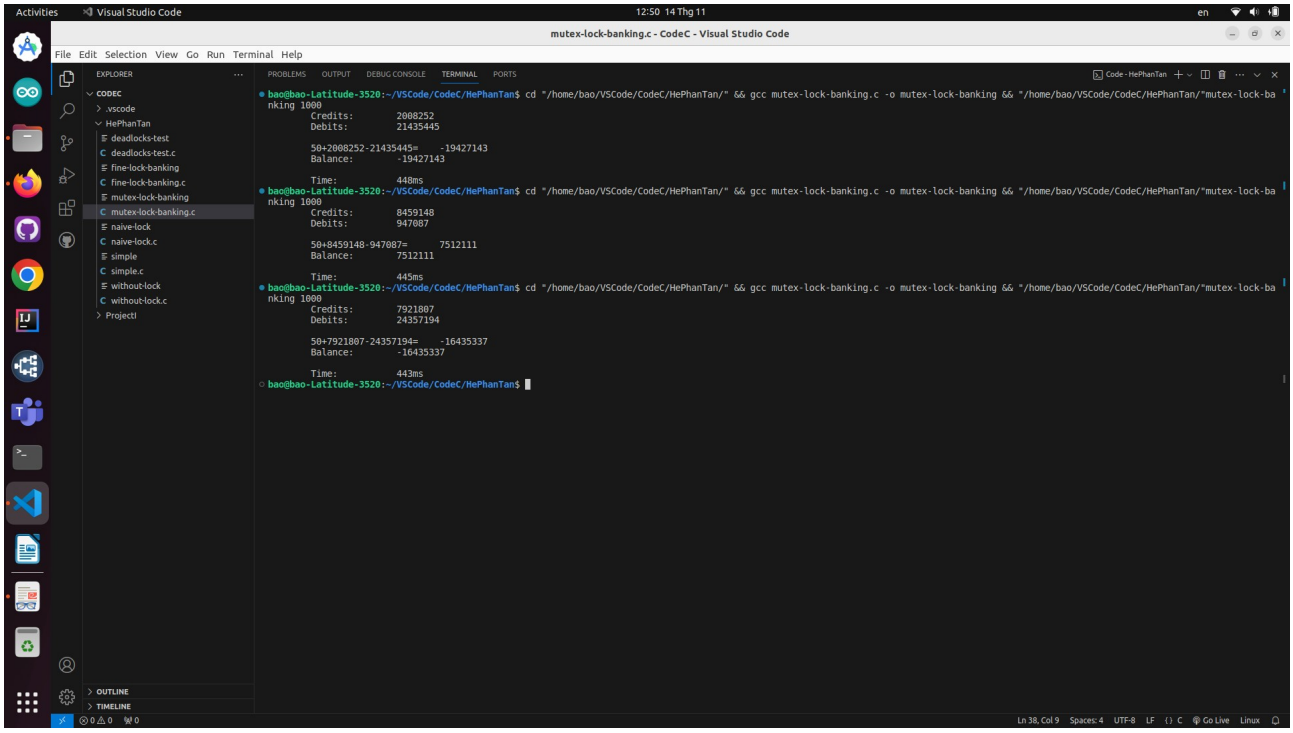
Vấn đề ở đây là nếu hai luồng cùng cố gắng truy cập vào biến `shared` cùng lúc, thì cả hai luồng đều sẽ chờ nhau. Điều này sẽ dẫn đến tình trạng deadlock, nghĩa là không luồng nào có thể tiếp tục thực hiện

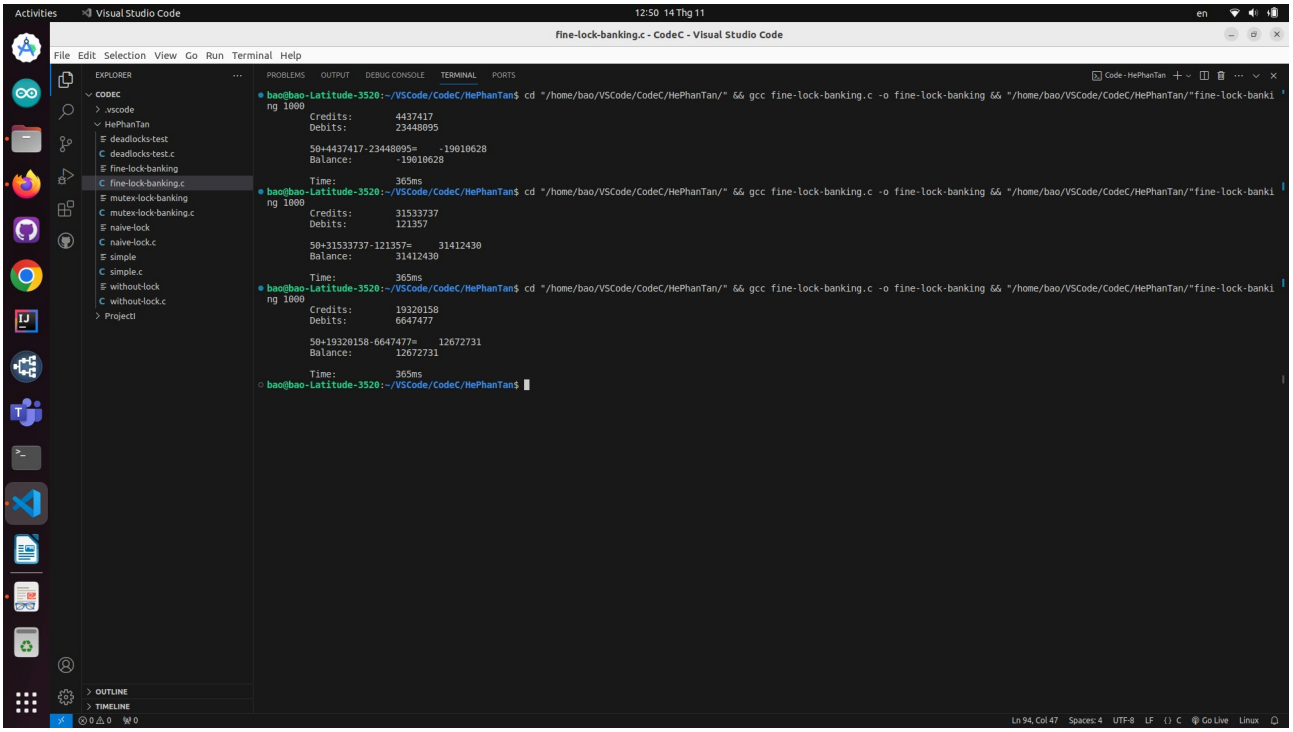
Câu hỏi 7: Bây giờ hãy thay đổi đoạn code của file without-lock.c bằng cách triển khai cơ chế mutex lock như trên (bạn có thể tạo file mới và đặt tên khác đi như mutex-lock-banking.c). Chạy chương trình nhiều lần và đánh giá đầu ra. Nó có cải thiện gì hơn so với naive-lock?



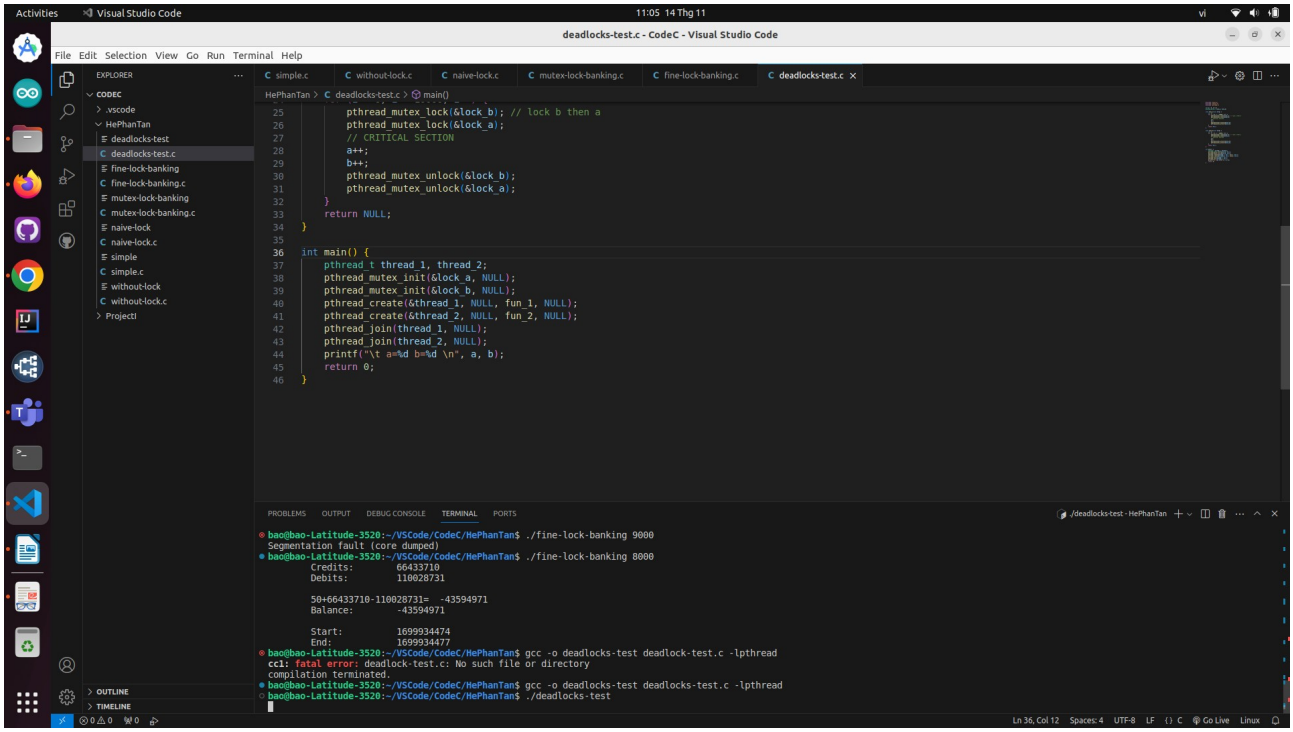
Không còn bị sai như naive-lock nữa.

Câu hỏi 8: So sánh và đo đạt thời gian để chứng minh là Fine Locking sẽ nhanh hơn Coarse Locking.





Câu hỏi 9: Chạy chương trình trên và bạn nhận thấy điều gì? Giải thích thông qua việc phân tích mã nguồn.



Chương trình không chạy được. Vì: Tiến trình 1 khóa a, tiến trình 2 khóa b cùng 1 lúc, khi 1 yêu cầu b và 2 yêu cầu a sẽ xảy ra deadlocks.